ANEXO nº2

Se crea la capa "bronze" del Data Lake en HDFS, diferenciando entre IPV e IPVA, y se cargan los ficheros JSON obtenidos del INE.

```
!hdfs dfs -mkdir -p /aula_B0.476/xrengel/datalake/bronze/ipv
!hdfs dfs -mkdir -p /aula_B0.476/xrengel/datalake/bronze/ipva

!hdfs dfs -put -f /home/xrengel/vivienda/IPV.json  /aula_B0.476/xrengel/datalake/bronze/ipv/
!hdfs dfs -put -f /home/xrengel/vivienda/IPVA.json /aula_B0.476/xrengel/datalake/bronze/ipva/
```

Se inicia una sesión en Apache Spark para poder cargar, transformar y almacenar los datos del proyecto.

```
import findspark
findspark.init()

import pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("local[*]") \
    .appName("Vivienda-IPV-IPVA") \
    .enableHiveSupport() \
    .getOrCreate()

sc = spark.sparkContext
```

Se leen los ficheros JSON y se cargan en Spark convirtiéndolos en "Dataframes", adaptando la lectura al formato de los JSON originales.

```
df_ipv_bronze = spark.read.option("multiline", "true").json(
    "hdfs:///aula_B0.476/xrengel/datalake/bronze/ipv/IPV.json"
)
df_ipva_bronze = spark.read.option("multiline", "true").json(
    "hdfs:///aula_B0.476/xrengel/datalake/bronze/ipva/IPVA.json"
)
```

Empezamos la capa "silver" con la transformación del "df_ipv_bronze".

Dado que se trata de una tabla con formato "multiline", lo primero que se debe hacer es un "explode", que convierte cada elemento del "array" en una fila independiente.

```
from pyspark.sql.functions import explode, col, avg, round, when, regexp_replace

df_ipv_exploded = df_ipv_bronze.withColumn("Data", explode("Data"))
```

Creamos un “Dataframe plano” seleccionando y renombrando las columnas que nos interesan.

```
df_ipv_flat = df_ipv_exploded.select(
    col("Nombre"),
    col("Data.Anyo").alias("Anyo"),
    col("Data.Valor").alias("Valor")
)
```

Como se puede observar a continuación, los nombres de las comunidades deben “limpiarse”.

```
df_ipv_flat.select("Nombre").distinct().show(truncate=False)
```

```
+-----------------------------------------------------+
|Nombre                                               |
+-----------------------------------------------------+
|Asturias, Principado de. General. Índice.            |
|País Vasco. General. Índice.                         |
|Ceuta. General. Índice.                              |
|Murcia, Región de. General. Índice.                  |
|Castilla y León. General. Variación anual.           |
|Cantabria. General. Índice.                          |
|Rioja, La. General. Índice.                          |
|Extremadura. General. Índice.                        |
|Cataluña. General. Variación anual.                  |
|Castilla y León. General. Índice.                    |
|Comunitat Valenciana. General. Variación anual.      |
|Madrid, Comunidad de. General. Variación anual.      |
|Murcia, Región de. General. Variación anual.         |
|Cataluña. General. Índice.                           |
|País Vasco. General. Variación anual.                |
|Aragón. General. Variación anual.                    |
|Cantabria. General. Variación anual.                 |
|Andalucía. General. Variación anual.                 |
|Galicia. General. Índice.                            |
|Navarra, Comunidad Foral de. General. Variación anual.|
+-----------------------------------------------------+
only showing top 20 rows
```

Se empieza dejando únicamente el nombre de la comunidad y quitando lo demás, aprovechando para cambiar el nombre de la columna a “Comunidad”.
Para poder diferenciar entre “Índice” y “Variación anual” creamos una nueva columna llamada “Tipo”.

```
from pyspark.sql.functions import regexp_replace, trim, col, when, concat_ws, split, udf
from pyspark.sql.types import StringType

df_ipv_clean = df_ipv_flat.withColumn(
    "Comunidad",
    trim(
        regexp_replace(
            regexp_replace(
                regexp_replace(col("Nombre"), r"\. Índice|\. Variación anual", ""),
                r"\. General", ""
```

```
            ),
            r"[.\s]+$", ""
        )
    )
).withColumn(
    "Tipo",
    when(col("Nombre").contains("Índice"), "ipv_total")
    .when(col("Nombre").contains("Variación anual"), "v_ipv_total")
)
```

En este punto se debe hacer otro proceso más, intercambiando el orden de las comunidades con una ”,” en su nombre, y eliminando el “.” que aparece al hacerlo. En la imagen se puede ver el resultado final.

```
from pyspark.sql.functions import col, when, split, reverse, concat_ws

df_ipv_clean = df_ipv_clean.withColumn(
    "Comunidad",
    when(
        col("Comunidad").contains(","),
        concat_ws(" ", reverse(split(col("Comunidad"), ", ")))
    ).otherwise(col("Comunidad"))
)
```

```
+--------------------------+
|Comunidad                 |
+--------------------------+
|Comunidad Foral de Navarra|
|Aragón                    |
|Comunitat Valenciana      |
|Cantabria                 |
|Extremadura               |
|Galicia                   |
|Ceuta                     |
|Illes Balears             |
|Principado de Asturias    |
|Canarias                  |
|Andalucía                 |
|Región de Murcia          |
|Castilla - La Mancha      |
|Total Nacional            |
|Cataluña                  |
|Castilla y León           |
|La Rioja                  |
|País Vasco                |
|Comunidad de Madrid       |
|Melilla                   |
+--------------------------+
```

Una vez “limpiados” los nombres, se “pivotea” en la columna “Tipo” creando dos nuevas columnas, “IPV_anual” y “V_IPV_anual”, con el valor anual resultado de la media de los 4 trimestres, dejando así un único registro por comunidad y año. Finalmente, aunque no es necesario, se ordena la tabla por comunidad y año para facilitar la lectura.

```
df_ipv_pivot = df_ipv_clean.groupBy("Comunidad",
"Anyo").pivot("Tipo").agg(round(avg("Valor"),2))

df_ipv_final = df_ipv_pivot.orderBy("Comunidad", "Anyo")
```

Replicamos el proceso para la tabla IPVA, pero al disponer ya los valores anuales no es necesario hacer la media de los trimestres.

```
from pyspark.sql.functions import col, when, split, reverse, concat_ws

from pyspark.sql.functions import (
    explode, col, regexp_replace, trim,
    when, concat_ws, expr
)

df_ipva_exploded = df_ipva_bronze.withColumn(
    "DataRow",
    explode(col("Data"))
)

df_ipva_base = df_ipva_exploded.select(
    col("Nombre"),
    col("DataRow.Anyo").alias("Anyo"),
    col("DataRow.Valor").alias("Valor")
)

df_ipva_clean = df_ipva_base.withColumn(
    "Comunidad",
    trim(
        regexp_replace(
            regexp_replace(
                regexp_replace(
                    col("Nombre"),
                    r"\. Índice|\. Variación anual", ""
                ),
                r"\. Total\.?", ""
            ),
            r"\.$", ""
        )
    )
)


df_ipva_clean = df_ipva_clean.withColumn(
    "Comunidad",
    when(
        col("Comunidad").contains(","),
        concat_ws(
            " ",
            expr("reverse(split(Comunidad, ', '))")
        )
```

```
    ).otherwise(col("Comunidad"))
)


df_ipva_clean = df_ipva_clean.withColumn(
    "Tipo",
    when(col("Nombre").contains("Índice"), "ipva")
    .when(col("Nombre").contains("Variación anual"), "v_ipva")
)

df_ipva_final = df_ipva_clean.groupBy(
    "Comunidad", "Anyo"
).pivot(
    "Tipo"
).agg(
    expr("first(Valor)")
)
```

Una vez arregladas ambas tablas se procede a unirlas. Después de renombrar las columnas de los índices y las variaciones para que tengan el mismo formato, se realiza el "join" de modo que cada fila contenga los datos de venta y alquiler de la misma comunidad y año.

```
df_ipv_ready = (
    df_ipv_final
    .withColumnRenamed("ipv_total", "ipv_anual")
    .withColumnRenamed("v_ipv_total", "v_ipv_anual")
)

df_ipva_ready = (
    df_ipva_final
    .withColumnRenamed("ipva", "ipva_anual")
    .withColumnRenamed("v_ipva", "v_ipva_anual")
)

df_final = (
    df_ipv_ready.alias("ipv")
    .join(
        df_ipva_ready.alias("ipva"),
        on=[
            col("ipv.Comunidad") == col("ipva.Comunidad"),
            col("ipv.Anyo") == col("ipva.Anyo")
        ],
        how="inner"
    )
    .select(
        col("ipv.Comunidad").alias("Comunidad"),
        col("ipv.Anyo").alias("Anyo"),
        col("ipv.ipv_anual"),
        col("ipva.ipva_anual"),
        col("ipv.v_ipv_anual"),
        col("ipva.v_ipva_anual")
```

```
    )
)
```

Para terminar, se añade el KPI propio "DIF_V" calculando la diferencia entre "v_ipva_anual" y "v_ipv_anual". Finalmente, se ordena la tabla y se guarda en la capa "gold" del "Datalake" en formatos JSON y Parquet.

```
df_final_gold = df_final.withColumn(
    "DIF_V",
    col("v_ipva_anual") - col("v_ipv_anual")
).orderBy(
    col("Comunidad"), col("Anyo")
)
```

Una vez terminado todo el proceso se comprueba que todo ha salido bien antes de guardar.

```
df_final_gold.toPandas()
```

	Comunidad	Anyo	ipv_anual	ipva_anual	v_ipv_anual	v_ipva_anual
0	Andalucía	2014	97.08	100.084	0.18	-0.6
1	Andalucía	2015	100.00	100.000	3.03	-0.1
2	Andalucía	2016	102.84	101.957	2.90	2.0
3	Andalucía	2017	106.10	103.593	3.18	1.6
4	Andalucía	2018	111.37	105.957	4.98	2.3
5	Andalucía	2019	117.36	109.594	5.38	3.4
6	Andalucía	2020	120.10	110.374	2.35	0.7
7	Andalucía	2021	125.60	112.471	4.58	1.9
8	Andalucía	2022	136.79	115.570	8.98	2.8
9	Andalucía	2023	143.57	118.530	4.95	2.6
10	Aragón	2014	99.16	100.470	-2.28	-1.5
11	Aragón	2015	100.00	100.000	0.85	-0.5
12	Aragón	2016	101.90	101.027	1.88	1.0
13	Aragón	2017	104.79	102.200	2.83	1.2
14	Aragón	2018	109.84	104.198	4.83	2.0
15	Aragón	2019	116.51	107.388	6.10	3.1
16	Aragón	2020	119.49	108.538	2.55	1.1
17	Aragón	2021	124.02	110.742	3.80	2.0
18	Aragón	2022	131.96	113.566	6.43	2.5
19	Aragón	2023	137.11	116.212	3.90	2.3
20	Canarias	2014	96.71	99.464	-1.25	0.1
21	Canarias	2015	100.00	100.000	3.40	0.5
22	Canarias	2016	103.34	101.982	3.38	2.0
23	Canarias	2017	107.89	104.312	4.38	2.3
24	Canarias	2018	113.28	107.050	4.98	2.6
25	Canarias	2019	117.59	110.732	3.83	3.4

Creamos los directorios “gold” en el “Datalake” de HDFS y guardamos la tabla final en dos formatos: el original, JSON, y en Parquet.

```
!hdfs dfs -mkdir -p /aula_B0.476/xrengel/datalake/gold/json
!hdfs dfs -mkdir -p /aula_B0.476/xrengel/datalake/gold/parquet

ruta_gold = "/aula_B0.476/xrengel/datalake/gold"

df_final_gold.write.mode("overwrite").json(ruta_gold + "/json")
df_final_gold.write.mode("overwrite").parquet(ruta_gold + "/parquet")
```

Para facilitar el trabajo y evitar posibles problemas, se decide exportar ambos ficheros.

```
df_final_gold.write.mode("overwrite").parquet("/tmp/gold_parquet_unico")
df_final_gold.write.mode("overwrite").json("/tmp/gold_json_unico")

!mkdir -p /home/xrengel/vivienda

!hdfs dfs -getmerge /aula_B0.476/xrengel/datalake/gold/parquet
/home/xrengel/vivienda/gold_parquet_final.parquet
!hdfs dfs -getmerge /aula_B0.476/xrengel/datalake/gold/json
/home/xrengel/vivienda/gold_json_final.json
```